Movie Ticketing System

Software Requirements Specification

Version 0.8

11/12/2026

Group 11
Cameron Penaranda, Justin Paul Liporada,
Ayaan Bhatkar

Prepared for
CS 250 - Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Fall 2026

Revision History

Date	Description	Author	Comments
9/22/2026	Version 0.1	Cameron Penaranda, Justin Paul Liporada, Ayaan Bhatkar	First revision,
9/24/2026	Version 0.2	Cameron Penaranda, Justin Paul Liporada, Ayaan Bhatkar	Second revision, completed functionalities and use cases.
10/08/2026	Version 0.3	Cameron Penaranda, Justin Paul Liporada, Ayaan Bhatkar	Added Software Design Specification
10/08/2026	Version 0.4	Cameron Penaranda	Refining earlier sections, removing redundancies.
10/08/2026	Version 0.5	Cameron Penaranda, Justin Paul Liporada, Ayaan Bhatkar	Cleaned appendices and areas the document hasn't covered yet. Refreshed table of contents.
10/08/2026	Version 0.6	Cameron Penaranda, Justin Paul Liporada, Ayaan Bhatkar	Added a third use case (Support Agent), described attributes, operations, and relationships in Section 6.3 instead of 3.4. Added SWA and UML diagrams and explanations.
10/22/2026	Version 0.7	Cameron Penaranda, Justin Paul Liporada, Ayaan Bhatkar	Added Test Plan section, verification and validation strategy, and test case documentation.

11/12/2026	Version 0.8	Cameron Penaranda, Justin Paul Liporada, Ayaan Bhatkar	Revised the SRS by expanding Section 7 Test Plan documentation with detailed testing strategies, representative test vectors, expected outputs, failure conditions, requirement-to-test case mapping, and additional test case documentation. Added Software Architecture Design updates including database components and expanded the Data Management Strategy to further describe database organization, design decisions, alternative approaches, and technology tradeoffs.

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	<Your Name>	Software Eng.	
	Dr. Gus Hanna	Instructor, CS 250	

REVISION HISTORY 1

DOCUMENT APPROVAL 3

1. INTRODUCTION 4

1.1 PURPOSE 5
1.2 SCOPE 5
1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS 5
1.4 REFERENCES 5
1.5 OVERVIEW 5

2. GENERAL DESCRIPTION 5

2.1 PRODUCT PERSPECTIVE 6
2.2 PRODUCT FUNCTIONS 6
2.3 USER CHARACTERISTICS 6
2.4 GENERAL CONSTRAINTS 6
2.5 ASSUMPTIONS AND DEPENDENCIES 7

3. SPECIFIC REQUIREMENTS 7

3.1 EXTERNAL INTERFACE REQUIREMENTS 8
3.1.1 User Interfaces 8
3.1.2 Hardware Interfaces 8
3.1.3 Software Interfaces 8
3.1.4 Communications Interfaces 8
3.2 FUNCTIONAL REQUIREMENTS 8
3.2.1 Browse movies 9
3.2.2 Locate Nearby Theaters 9
3.2.3 View showtimes 9
3.2.4 Select Seats 10
3.2.5 Browse Food and Drink 10
3.2.6 Purchase Tickets 11
3.2.7 Purchase Confirmation 11
3.2.8 Manage Movies 12
3.2.9 Manage Showtimes 12
3.2.10 Support Tickets and Customer Chat 13
3.3 USE CASES 13
3.3.1 Use Case #1 13
3.3.2 Use Case #2 13
3.4 CLASSES / OBJECTS 14
3.4.1 User 14
3.4.2 Customer 15
3.4.3 Administrator 15
3.4.4 Movie 15
3.4.5 Theater 15
3.4.6 Auditorium 15
3.4.7 Showtime 15
3.4.8 Seat 15
3.4.9 ConcessionItem 15
3.4.10 Order 15
3.4.11 Ticket 15
3.4.12 Payment 15
3.5 NON-FUNCTIONAL REQUIREMENTS 15
3.5.1 Performance 15
3.5.2 Reliability 16
3.5.3 Availability 16
3.5.4 Security 16

3.5.5 Maintainability 16
3.5.6 Portability 16
3.6 INVERSE REQUIREMENTS 16
3.7 DESIGN CONSTRAINTS 17
3.8 LOGICAL DATABASE REQUIREMENTS 17
3.9 OTHER REQUIREMENTS 17

4. ANALYSIS MODELS 18

4.1 SEQUENCE DIAGRAMS 18
4.3 DATA FLOW DIAGRAMS (DFD) 18
4.2 STATE-TRANSITION DIAGRAMS (STD) 18

5. CHANGE MANAGEMENT PROCESS 18

6. SOFTWARE DESIGN SPECIFICATION 18

6.1 SYSTEM DESCRIPTION 18
6.2 SOFTWARE ARCHITECTURE OVERVIEW 18
6.2.1 Software Architecture Diagram 19
6.2.2 Architecture Description 20
6.3 UML CLASS DESIGN 21
6.3.1 UML Class Diagram 21
6.3.2 User 22
6.3.3 Customer 23
6.3.4 Administrator 23
6.3.5 Movie24
6.3.6 Theater 24
6.3.7 Auditorium 25
6.3.8 Showtime 25
6.3.9 Seat 26
6.3.10 ConcessionItem 26
6.3.11 Order 27
6.3.12 Ticket 28
6.3.13 Payment 29
6.3.14 Class Relationships 29
6.4 DEVELOPMENT PLAN AND TIMELINE 29
6.4.1 Task Partitioning 30
6.4.2 Team Member Responsibilities 30
6.5 Data Management Strategy 30
6.5.1 Database Selection 30
6.5.2 Logical Organization of Data 31
6.5.3 Data Integrity and Storage Rules 32
6.5.4 Seat Reservations and Payment Processing 32
6.5.5 Security and Access Control 33
6.5.6 Backup, Recovery, and Retention 33
6.5.7 Alternatives and Tradeoffs 34

7. TEST PLAN 34

7.1 TESTING OBJECTIVES 35
7.2 VERIFICATION AND VALIDATION STRATEGY 35
7.3 UNIT TESTING 36
7.4 FUNCTIONAL TESTING 36
7.5 SYSTEM TESTING / INTEGRATION TESTING 37
7.5.1 Scenario 1 – Complete Ticket Purchase 37
7.5.2 Scenario 2 – Double Seat Reservation 38
7.6 TEST COVERAGE 38
7.7 TEST ENVIRONMENT 38
7.8 TEST CASE DOCUMENT 39

7.9 GITHUB REPOSITORY 39

A. APPENDICES 39

A.1 APPENDIX 1 40
A.2 APPENDIX 2 40

1. Introduction

1.1 Purpose

The purpose of this SRS document is to define the functional and non-functional requirements of the Movie ticketing system. The document establishes what the system is expected to do, how users will interact with it, what external interfaces may be required, and what constraints must be considered during development. This SRS is intended to provide an understanding of the Movie ticketing system for the development team, testers, project stakeholders, and the CS250 instructor. Developers can use the requirements to help guide them through the implantations, while testers can use them to verify that the completed system satisfies the specified requirements.

1.2 Scope

The Movie Ticketing System is a software system that allows customers to create an account, browse available movies from all movie theaters, available showtimes and auditoriums for currently airing movies, seat selections, snacks for the movie, and purchasing of everything through credit, debit, and PayPal. The user's receipt of sale is then shown to them and has the option to send a copy of their receipt to their email address with possible refunds. The Movie Ticketing System will not accept EBT.

1.3 Definitions, Acronyms, and Abbreviations

SRS	Software Requirement Specifications
EBT	Electronic Benefits Transfer
CS250	Introduction to Software Systems
API	Application Programming Interface
GUI	Graphical User Interface
UML	Unified Modeling Language
SWA	Software Architecture
GPS	Global Positioning System
UTC	Coordinated Universal Time

1.4 References

CS 250 Software Requirements Specification Template, Fall 2026.
CS 250 course lecture materials and assignment instructions.

1.5 Overview

The remainder of this SRS describes the Movie Ticketing System's general characteristics, functional requirements, external interfaces, use cases, classes and objects, non-function requirements, database requirements, analysis models, and change management process.

2. General Description

2.1 Product Perspective

The Movie Ticketing System is intended to provide a centralized application that customers can locate nearby participating movie theaters, browse available movies and showtimes, select seats, purchase movie tickets, and purchase available food and drink items. The system will maintain information concerning theaters, theater locations, movies, showtimes, seating arrangements, concession items, and customer purchases. Location services may be used to determine the customer's current location and identify nearby theaters. If location access is unavailable or denied, customers will be able to search for theaters manually. The system may also interact with an external payment processing service when completing customer purchases. Administrators will have access to separate functionality for maintaining movie, showtime, and concession information.

2.2 Product Functions

The Movie Ticketing System will provide customers with the ability to search for movies. After selecting a movie, customers will be able to view movie information and the available showtimes at various theaters and be able to select a showtime for ticket purchase. The system will display the seating arrangement for the selected showtime and allow customers to select available seats. Customers will also be able to browse food and drink items offered by the selected theater and add available concession items to their order. Before completing a purchase, the system will calculate and display the total cost of the selected tickets and concession items. After a successful transaction, the system will generate purchase confirmation information. Authorized administrators will be able to manage movie information, showtimes, and concession offerings. Authorized support agents will be able to review customer support tickets, communicate with customers through chat, and update ticket statuses.

2.3 User Characteristics

The primary users of the Movie Ticketing System are customers and administrators. Customers are expected to have basic familiarity with using a computer, smartphone, or similar electronic device, but they will not require specialized technical knowledge to use the system. Customers will use the application to search for theaters, browse movies and showtimes, select seats, purchase tickets, browse food and drink items, and complete orders. Administrative users are expected to have basic knowledge of theater operations and the ability to operate a graphical software application. Administrators will be responsible for maintaining information such as available movies, scheduled showtimes, and concession offerings. Administrative functionality will be restricted to authorized users and will not be accessible to ordinary customers. Support Agents are authorized users that will help users with questions. They are responsible for reviewing and responding customer support tickets. The Support Agent functionality will be restricted to authorized users and will not be accessible to ordinary customers.

2.4 General Constraints

The Movie Ticketing System must be developed within the schedule and requirements established for the CS 250 project and must provide a graphical user interface through which customers and administrators can interact with the system. The system must maintain persistent information concerning participating theaters, movies, showtimes, auditoriums, seats, concession items, and completed purchases. Location-based theater discovery will depend on the customer's device supporting location services and the customer granting permission to access that information. Because location permission may be denied or unavailable, the system must also support manual theater searching. Each showtime must correspond to a specific movie, theater, auditorium, date, and time, and the same seat must not be sold to multiple customers for the same showtime. Movie, showtime, and food and drink availability may differ between participating theaters, and administrative functions must only be accessible to authorized administrators.

2.5 Assumptions and Dependencies

The Movie Ticketing System assumes that customers have access to a compatible computing device and the network connection necessary to use the application. Participating theaters are assumed to provide accurate information regarding their locations, available movies, scheduled showtimes, auditorium seating arrangements, and available food and drink items. Each theater is assumed to contain one or more defined auditoriums, and each auditorium is assumed to contain a predefined collection of seats. Each showtime is assumed to be associated with a valid movie, theater, and auditorium. Location-based functionality depends on location services supplied by the customer's device, browser, or operating system, although customers may manually provide a location if these services are unavailable or permission is denied. The system also depends on persistent data storage for maintaining theater, movie, showtime, seat, concession, and purchase information. If an external payment processing service is used, successful online purchasing will additionally depend on the availability of that service. These assumptions represent external conditions that may cause the system requirements to change if they are no longer valid.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The Movie Ticketing System shall provide a graphical user interface in which customers can browse movies, view showtimes, select seats, purchase tickets, and view confirmations.
The customer interface will include:

- Theater discovery
- Theater search
- Movie browsing
- Movie information
- Showtime selection
- Seat selection
- Food and drink browsing
- Order summary
- Checkout
- Purchase confirmation
- Support ticket dashboard
- List of submitted support tickets
- Ticket details and customer information
- Customer chat interface
- Chat conversation history
- Ticket status management

The location interface shall request permission before the user's device location. The system will provide a manual theater-search interface when location service is unavailable or denied.

3.1.2 Hardware Interfaces

The Movie Ticketing System does not require specialized hardware interfaces. The system will operate on standard computing devices capable of running a supported modern web browser. The system may indirectly interact with device hardware used by location services, such as GPS, through browser or operating system location functionality.

3.1.3 Software Interfaces

The Movie Ticketing System shall interface with a persistent database for storing and retrieving movie, theater, showtime, seat, customer, and ticket information. The system may also interface with an external payment processing service for payment authorization.

3.1.4 Communications Interfaces

The Movie Ticketing System shall communicate between the client application and the server using network-based communication.

Communication involving sensitive information shall use secure network communication.

3.2 Functional Requirements

3.2.1 Browse movies

3.2.1.1 Introduction
The system shall allow customers to browse movies currently available for ticket purchase.

3.2.1.2 Inputs
The system shall accept a customer's request to view available movies.

3.2.1.3 Processing
The system shall retrieve movies currently available from persistent storage.

3.2.1.4 Outputs
The system shall display each available movie's title and associated movie information.

3.2.1.5 Error Handling
If no movies are available, the system shall notify the customer that no movies are currently available.

3.2.2 Locate Nearby Theaters

3.2.2.1 Introduction
The system shall allow customers to locate participating movie theaters near their current location.

3.2.2.2 Inputs
The system shall allow the user to request nearby theaters.
The system shall request permission before accessing the user's current location.
The system shall allow the user to manually enter a search location.

3.2.2.3 Processing
When location permission is granted, the system shall obtain the user's current location through supported location services.
The system shall compare the search location with participating theater locations.

3.2.2.4 Outputs
The system shall display participating theaters located near the user's requested location.
The system shall allow the customer to select a displayed theater.

3.2.2.5 Error Handling
If location permission is denied, the system shall provide manual theater search.
If no participating movie theaters are found for the requested location, the system shall notify the customer.
Failure of location services shall not prevent the user from manually searching for theaters.

3.2.3 View showtimes

3.2.3.1 Introduction
The system shall allow customers to view available showtimes for a selected movie.

3.2.3.2 Inputs
The customer shall select a movie.

3.2.3.3 Processing
The system shall retrieve all valid showtimes associated with the selected movie.

3.2.3.4 Outputs
The system shall display the date and start time associated with the selected movie.

3.2.3.5 Error Handling
If the selected movie has no scheduled showtimes, the system shall notify the customer that there are no available showtimes available.

3.2.4 Select Seats

3.2.4.1 Introduction
The system shall allow customers to select seats for a selected movie showtime.

3.2.4.2 Inputs
The system shall accept the customer's selection of one or more available seats.

3.2.4.3 Processing
The system shall retrieve the seating arrangement for the selected showtime and determine which seats are currently available.

3.2.4.4 Outputs
The system shall retrieve the seating arrangement and distinguish available seats from unavailable seats.

3.2.4.5 Error Handling
If a customer attempts to select an unavailable seat, the system shall notify the customer that the seat is unavailable and require another seat to be selected.

3.2.5 Browse Food and Drink

3.2.5.1 Introduction
The system shall allow customers to browse food and drink items available at the selected theater.

3.2.5.2 Inputs
The customer shall select a theater before browsing theater-specific concession items.

3.2.5.3 Processing

The system shall retrieve food and drink items available at the selected theater.

3.2.5.4 Outputs
The system shall display food and drink items.
The system shall display the price of concession items.
The system shall allow the user to add concession items to the current order.
The system shall allow the user to change the quantity of concession items in their current order.

3.2.5.5 Error Handling
The system shall prevent customers from ordering concession items that are unavailable.
If there are no concession items available at the participating theater, the system shall notify the user.

3.2.6 Purchase Tickets

3.2.6.1 Introduction
The system shall allow customers to purchase tickets for a selected movie showtime.

3.2.6.2 Inputs
The system shall accept the customer's selected showtime, selected seats, and required payment information.

3.2.6.3 Processing
The system shall verify that the selected seats are available, calculate the total ticket price, and process the customer's payment.

3.2.6.4 Outputs
The system shall display the total purchase price and whether the ticket purchase was successfully completed.

3.2.6.5 Error Handling
If payment processing fails for a selected seat or is no longer available, the system shall notify the customer and shall not complete the purchase.

3.2.7 Purchase Confirmation

3.2.7.1 Introduction
The system shall provide the customer with confirmation after a successful ticket purchase.

3.2.7.2 Inputs
The system shall receive the information associated with the successfully completed purchase.

3.2.7.3 Processing
The system shall generate a unique confirmation code for the completed purchase and associate it with the purchased tickets.

3.2.7.4 Outputs
The system shall display the movie title, theater, showtime, selected seats, purchase total, and confirmation information.

3.2.7.5 Error Handling
If purchase confirmation cannot be generated or displayed, the system shall notify the customer that an error occurred while retrieving the purchase confirmation.

3.2.8 Manage Movies

3.2.8.1 Introduction
The system shall allow authorized administrators to manage movie information.

3.2.8.2 Inputs
The system shall allow authorized administrators to accept, modify, or remove information.

3.2.8.3 Processing
The system shall validate the movie information and update the stored movie data according to an authorized administrator's requested action.

3.2.8.4 Outputs
The system shall display the updated movie information and confirm that the requested change was completed.

3.2.8.5 Error Handling
If the movie information is incomplete or invalid, the system shall notify the administrator and shall not save the changes until the error is corrected.

3.2.9 Manage Showtimes

3.2.9.1 Introduction
The system shall allow authorized administrators to manage movie showtimes.

3.2.9.2 Inputs
The system shall accept support tickets from customers and messages from them through a chat for support agents to manage.

3.2.9.3 Processing
The system shall store support tickets and associate each ticket with the customer who submitted it. The system shall allow support agents to manage support tickets with the history of the issue and chats maintained.

3.2.9.4 Outputs
The system shall display opening and closing these issues are done by support agents.

3.2.9.5 Error Handling

If the support was not able to be sent, the customer will get a message. If chat with a support agent is not currently available, the customer will know.

3.2.10 Support Tickets and Customer Chat

3.2.10.1 Introduction
The system shall allow authorized support agents to manage customer chats and support tickets.

3.2.10.2 Inputs
The system shall accept showtime information entered, modified, or removed by an authorized administrator.

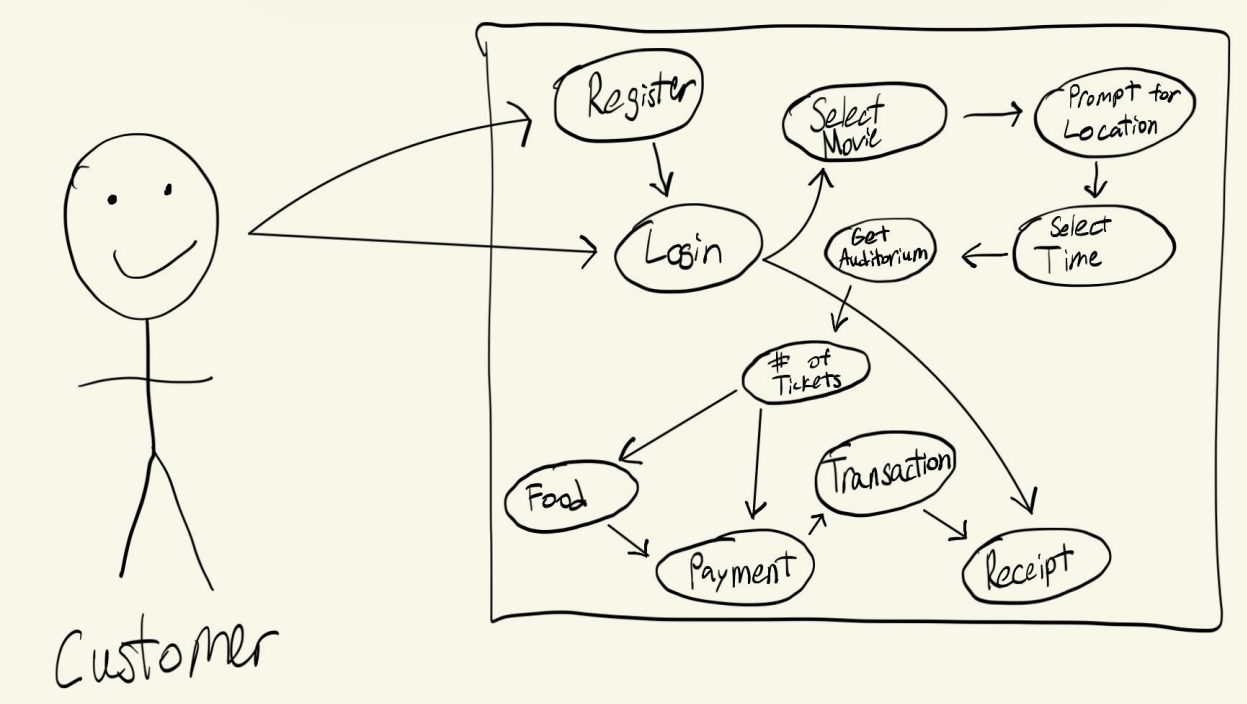
3.2.10.3 Processing
The system shall validate the showtime information and update the stored showtime data according to the authorized administrator's requested action.

3.2.10.4 Outputs
The system shall display the updated movie showtime information and confirm that the requested change was completed.

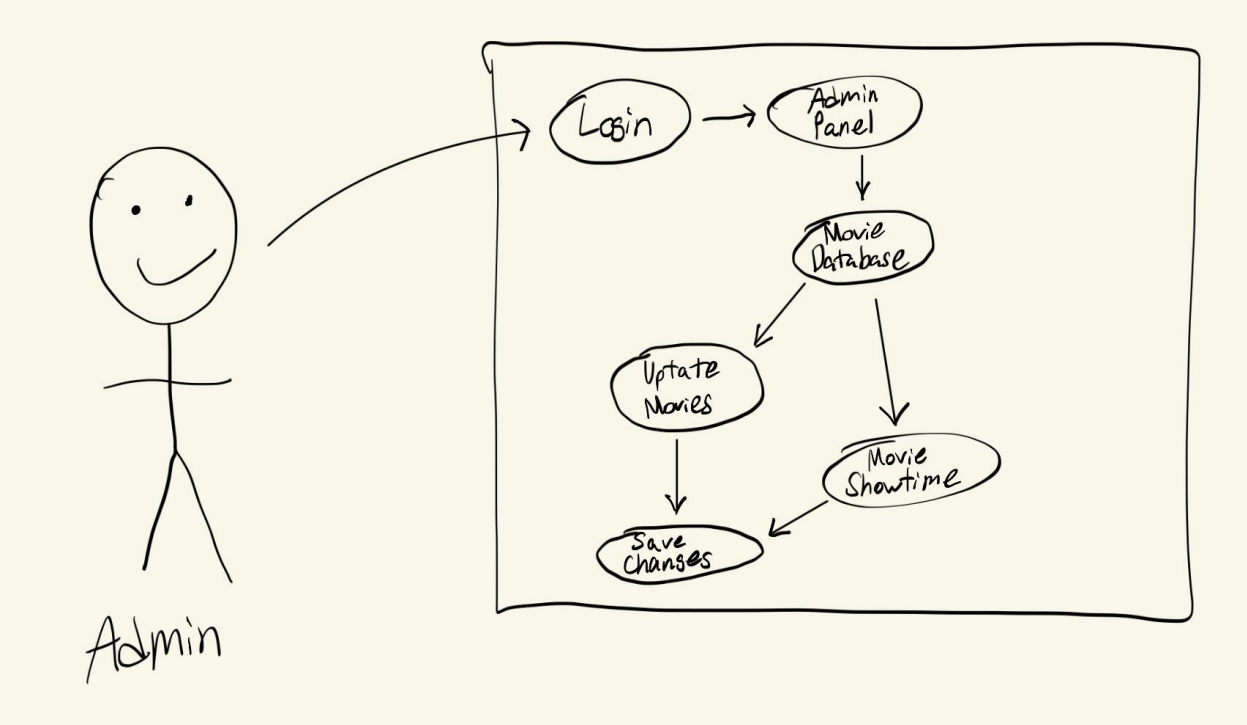
3.2.10.5 Error Handling
If the showtime information is incomplete, invalid, or conflicts with an existing showtime, the system shall notify the administrator and shall not save the changes until the error is corrected.

3.3 Use Cases

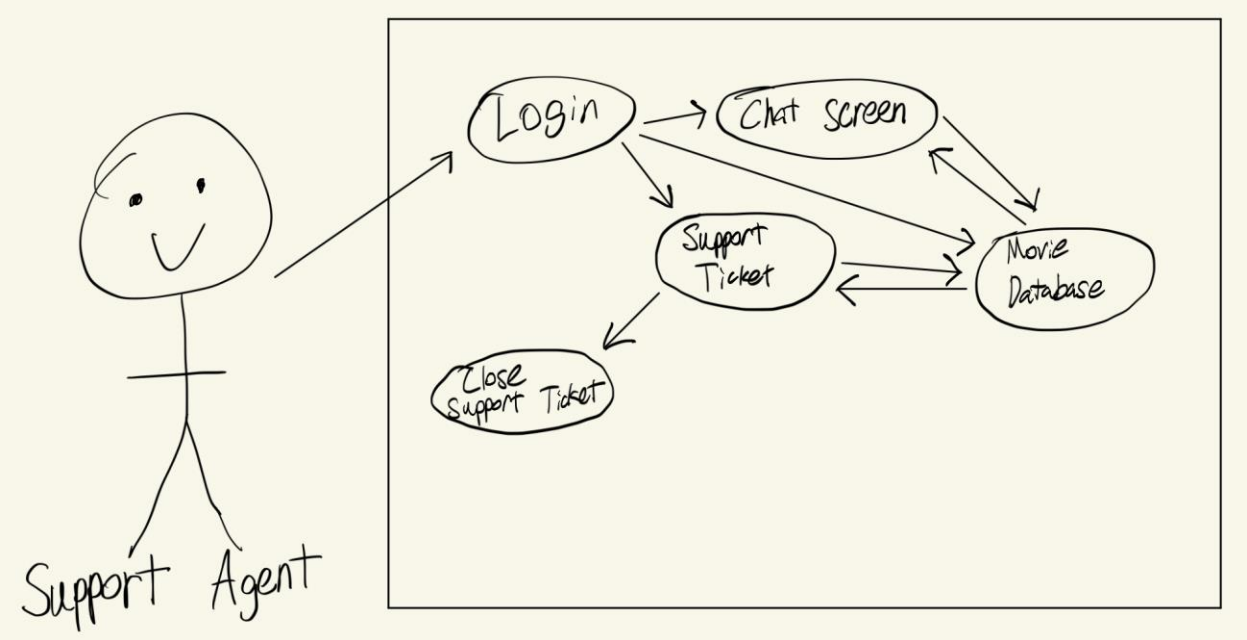
3.3.1 Use Case #1



3.3.2 Use Case #2



3.3.3 Use Case #3



3.4 Classes / Objects

The classes below identify the main account, theater, scheduling, and purchase concepts required by the Movie Ticketing System. Detailed attributes, operations, and relationships are described in Section 6.3.

3.4.1 User

Represents a registered account and provides the username, password information, email address, and login and logout functionality.

3.4.2 Customer

Inherits account information from User and creates and views customer orders.

3.4.3 Administrator

Inherits account information from User and manages movie and showtime information.

3.4.4 Movie

Stores a movie's title, runtime, age rating, genre, and description and provides access to its details and showtimes. Theater, auditorium, and seat information is represented by separate classes.

3.4.5 Theater

Represents a participating theater, including its name, address, and geographic location, and provides its movies, showtimes, and concession offerings.

3.4.6 Auditorium

Represents a screening room within a theater, including its number, capacity, and seats.

3.4.7 Showtime

Associates a movie with an auditorium and a scheduled date and start time and provides available-seat information.

3.4.8 Seat

Identifies a seat within an auditorium by its row and number and determines availability for a selected showtime.

3.4.9 ConcessionItem

Represents a food or drink item offered by a theater, including its name, price, and availability.

3.4.10 Order

Groups a customer's tickets and concession items, maintains the purchase total and order status, and supports adding, removing, and purchasing the selected items.

3.4.11 Ticket

Represents a purchased reservation for a specific showtime and seat and stores its price and status.

3.4.12 Payment

Records an order's payment amount, method, and status and supports payment processing and refunds.

3.5 Non-Functional Requirements

3.5.1 Performance

- Timely responses to customers actions:
    - Login will be completed in 5 seconds most in normal operating conditions

- System should display all available movies in 3 seconds most in normal operating conditions
- System will display all available showtimes form different theatres for the selected movie in 3 seconds most in normal operating conditions
- System will display the seating information for a selected showtime in 3 seconds most in normal operating conditions
- System will display options to add snacks and beverages 2 seconds most
- System will display all available snacks and beverages available at selected theatre location n 3 seconds most in normal operating conditions
- System will provide a payment result or indicate the payment is still processing within 10 seconds at most after the customer submits the required payment information in normal operating conditions
- After a successful payment authorization, the system should generate and display the ticket confirmation in 5 seconds at most in normal operating conditions

3.5.2 Reliability

- System will not generate a valid ticket unless payment has been successfully authorized
- The system will not sell the same seat to more than one customer for the same showtime
- The system will preserve confirmed ticket purchases in the event of a system error or temporary interruption

3.5.3 Availability

- System should not remain unavailable for longer than 1 continuous hour
- Planned maintenance will be communicated to users 24 hours before system becomes unavailable
- System gives 20 minutes for selecting and purchasing seats before ending transaction and making seats selectable again

3.5.4 Security

- Sensitive payment information shall be transmitted securely
- System will not display a customer's full card number after payment has been submitted
- System will prevent unauthorized users from accessing another customer's account or ticket information
- User passwords will not be displayed in plain text

3.5.5 Maintainability

- Movie, theatre, showtime, snack and beverage information shall be updateable without requiring changes to the system’s source code
- The system’s source code will be maintained using a version control system

3.5.6 Portability

- The system will operate on all modern web browsers
- The system will support both desktop and mobile screen sizes
- All core functions and data will remain available and saved across platforms through user login

3.6 Inverse Requirements

*State any *useful* inverse requirements.*

3.7 Design Constraints

Specify design constrains imposed by other standards, company policies, hardware limitation, etc. that will impact this software project.

3.8 Logical Database Requirements

The system shall use persistent relational storage for user accounts, movies, theaters, auditoriums, seats, showtimes, concessions, orders, payments, admission tickets, support tickets, and chat messages. Records shall use unique identifiers and maintain valid relationships. The system shall prevent duplicate active reservations for the same seat and showtime, preserve confirmed purchases, and store monetary amounts using fixed decimal values. Sensitive information shall be protected through password hashing, access restrictions, and secure communication. The detailed database design, backup procedures, and retention policies are described in Section 6.5.

3.9 Other Requirements

Catchall section for any additional requirements.

4. Analysis Models

List all analysis models used in developing specific requirements previously given in this SRS. Each model should include an introduction and a narrative description. Furthermore, each model should be traceable the SRS's requirements.

4.1 Sequence Diagrams

4.3 Data Flow Diagrams (DFD)

4.2 State-Transition Diagrams (STD)

5. Change Management Process

Identify and describe the process that will be used to update the SRS, as needed, when project scope or requirements change. Who can submit changes and by what means, and how will these changes be approved.

6. Software Design Specification

6.1 System Description

The Movie Ticketing System is a web based application that allows customers to browse movies, search for nearby theaters, view showtimes, select seats, order food and drinks, and purchase tickets. Customers can also create an account so that their purchases and ticket information can be connected to them. The system supports payment by credit card, debit card, and PayPal. After a purchase is completed, the customer is shown a receipt and can choose to have a copy sent to their email. These features match the scope already defined in the requirements section.

The system also includes administrator functions. Administrators are able to manage movie information and showtimes that appear in the application. Theater, movie, showtime, seat, concession, customer, and purchase information will be stored so that it can be retrieved when needed. The system also uses location services to help customers find nearby theaters, with manual search available if location access is not available.

6.2 Software Architecture Overview

The Movie Ticketing System will be divided into three main parts: the user interface, the application logic, and the database. Separating the system this way makes it easier to organize the code and keep different responsibilities from being mixed together.

The user interface is the part of the system that customers, administrators, and support agents interact with directly. It will contain the pages or screens for login, movie browsing, theater selection, showtime selection, seat selection, food and drink selection, checkout, purchase

confirmation, and customer support features. Administrators will also have access to interfaces for managing movie, showtime, and concession information. Support agents will have access to interfaces for reviewing customer support tickets, communicating with customers through chat, and updating ticket statuses.

The application logic will handle the actions requested through the user interface. For example, when a customer selects a movie, the application will retrieve the available showtimes. When a customer selects seats, the application will check whether the seats are still available. It will also calculate order totals, process purchases, create purchase confirmation information, and enforce system rules such as preventing duplicate seat reservations.

The application logic will also manage communication between the user interface, database, and external services. This layer will validate user permissions, apply business rules, process customer requests, and determine whether requested actions are allowed before updating stored information.

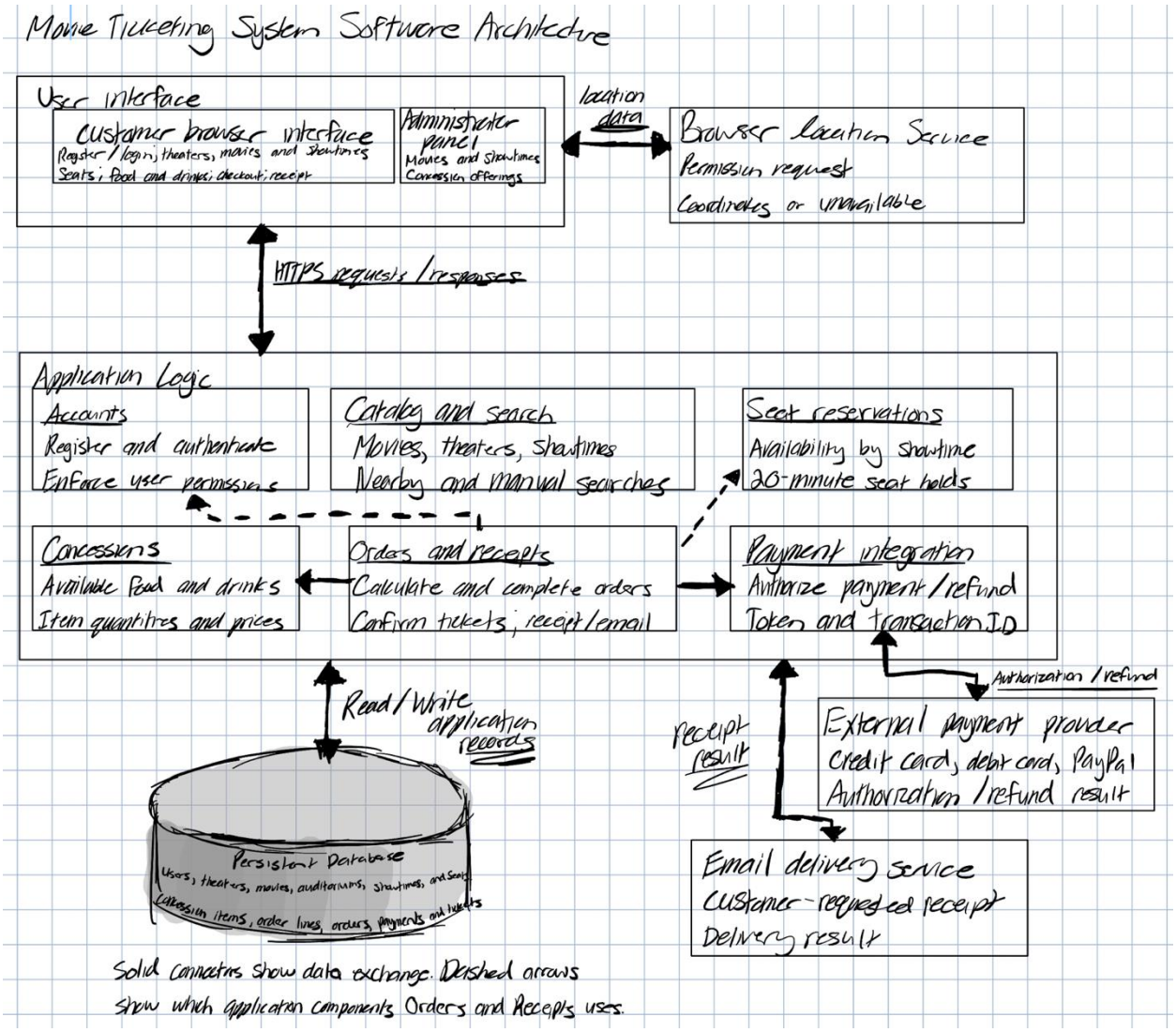
The database will store the information that needs to remain available between sessions. This includes user accounts, movies, theaters, auditoriums, showtimes, seats, concession items, orders, payments, tickets, support tickets, and chat messages. The application logic will communicate with the database whenever information needs to be retrieved or updated.

The Movie Ticketing System will use PostgreSQL as the primary database system because the information stored by the application contains many connected relationships. For example, customers create orders, orders contain tickets, and tickets reference specific seats and showtimes. A relational database allows these relationships to be maintained using tables, primary keys, and foreign keys.

The system may also communicate with outside services. Location services may be used to determine the customer's current location when searching for nearby theaters. A payment service may be used to authorize customer payments. Sensitive payment information will not be stored within the Movie Ticketing System database.

The system will also include a support agent interface that allows authorized support agents to review customer support tickets, communicate with customers through chat, and update ticket statuses. The application logic will manage support tickets and chat messages, while the database will store ticket information and conversation history.

6.2.1 Software Architecture Diagram



6.2.2 Architecture Description

The architecture separates the browser user interface, application logic, and persistent database. Customer, administrator, and support agent pages send requests to the application logic layer. The application logic validates requests, applies permissions and business rules, accesses stored records, and returns results to the appropriate user interface.

The user interface contains customer screens for registration and login; theater, movie, and showtime browsing; seat selection; concession selection; checkout; purchase confirmation; and support communication. The administrator panel supports movie, showtime, and concession maintenance. The support agent interface allows authorized support agents to review customer issues, communicate through chat, and update support ticket statuses.

The Accounts component registers and authenticates users and enforces customer, administrator, and support agent permissions. The Catalog and Search component retrieves movies, theaters, showtimes, and theater locations. The Seat Reservation component checks seat availability and manages temporary seat holds. The Concessions component retrieves theater-specific food and drink offerings and validates availability. The Orders and Receipts component manages selected items, calculates totals, processes purchases, and generates confirmations. The Payment Integration component communicates with the external payment provider for authorization and refunds.

The application logic reads and writes persistent information through database operations. The PostgreSQL database stores the entities required by the system, including users, movies, theaters, auditoriums, seats, showtimes, concession items, orders, tickets, payments, support tickets, and chat messages. Database relationships ensure that connected information remains consistent.

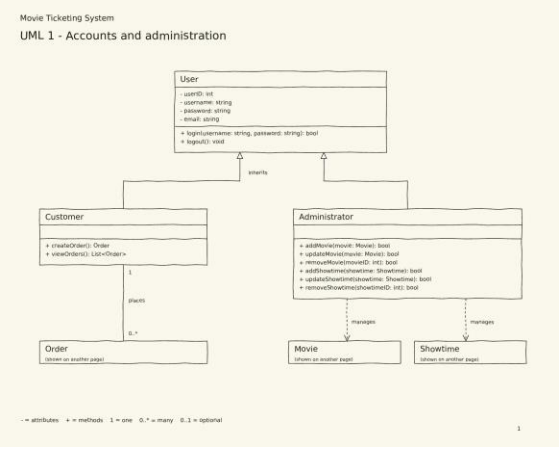
The database design prevents invalid operations by enforcing relationships between records. For example, a ticket must reference a valid order, showtime, and seat. The system also prevents multiple customers from purchasing the same seat for the same showtime.

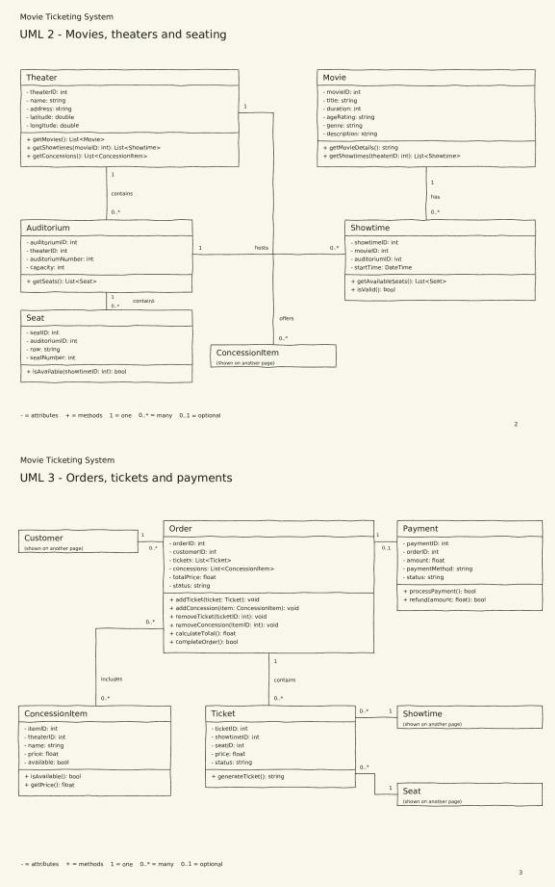
The browser location service returns coordinates or an unavailable result to the customer interface. These coordinates or manually entered locations are passed to the theater search functionality. Payment integration exchanges payment authorization information with the external payment provider while avoiding storage of sensitive payment details.

6.3 UML Class Design

The class design below defines the main classes, attributes, operations, and relationships used by the Movie Ticketing System.

6.3.1 UML Class Diagram





6.3.2 User

The `User` class represents a registered account in the system.

Attributes

`userID: int`
Stores the unique ID assigned to the user.

`username: string`
Stores the username used to log into the account.

`password: string`
Stores the user's password information.

`email: string`
Stores the email address connected to the account.

Operations

`login(username: string, password: string): bool`
Checks the username and password and returns true if the login is successful.

`logout(): void`
Logs the user out of the system.

6.3.3 Customer

The `Customer` class represents a customer using the system to purchase tickets and concessions. Customer will inherit basic account information from the `User` class.

Operations

`createOrder(): Order`
Creates a new order for the customer.

`viewOrders(): List<Order>`
Returns the customer's current and previous orders.

6.3.4 Administrator

The `Administrator` class represents a user with permission to manage movie and showtime information. Administrator will also inherit account information from the `User` class.

Operations

`addMovie(movie: Movie): bool`
Adds a new movie to the system.

`updateMovie(movie: Movie): bool`
Updates information for an existing movie.

`removeMovie(movieID: int): bool`
Removes a movie from the system.

`addShowtime(showtime: Showtime): bool`
Adds a new showtime.

`updateShowtime(showtime: Showtime): bool`
Updates an existing showtime.

`removeShowtime(showtimeID: int): bool`
Removes an existing showtime.

6.3.5 Movie

The `Movie` class stores information about a movie available in the system.

Attributes

`movieID: int`
Stores the unique ID for the movie.

`title: string`
Stores the movie title.

`duration: int`
Stores the movie runtime in minutes.

`ageRating: string`
Stores the age rating of the movie.

`genre: string`
Stores the genre of the movie.

`description: string`
Stores a short description of the movie.

Operations

`getMovieDetails(): string`
Returns the information stored for the movie.

`getShowtimes(theaterID: int): List<Showtime>`
Returns the available showtimes for the movie at a selected theater.

6.3.6 Theater

The `Theater` class represents a participating movie theater.

Attributes

`theaterID: int`
Stores the unique ID for the theater.

`name: string`
Stores the theater name.

`address: string`
Stores the theater address.

`latitude: double`
Stores the latitude of the theater.

`longitude: double`
Stores the longitude of the theater.

Operations

`getMovies(): List<Movie>`
Returns the movies currently available at the theater.

`getShowtimes(movieID: int): List<Showtime>`
Returns showtimes for a selected movie.

`getConcessions(): List<ConcessionItem>`
Returns the food and drink items available at the theater.

6.3.7 Auditorium

The `Auditorium` class represents a screening room inside a theater.

Attributes

`auditoriumID: int`
Stores the unique ID for the auditorium.

`theaterID: int`
Stores the ID of the theater that contains the auditorium.

`auditoriumNumber: int`
Stores the number assigned to the auditorium.

`capacity: int`
Stores the number of seats in the auditorium.

Operations

`getSeats(): List<Seat>`
Returns the seats assigned to the auditorium.

6.3.8 Showtime

The `Showtime` class represents a scheduled showing of a movie.

Attributes

`showtimeID: int`
Stores the unique ID for the showtime.

`movieID: int`
Stores the ID of the movie being shown.

`auditoriumID: int`
Stores the ID of the auditorium where the movie will be shown.

`startTime: DateTime`
Stores the date and starting time of the showing.

Operations

`getAvailableSeats(): List<Seat>`
Returns the seats that are currently available for the showtime.

`isValid(): bool`
Checks whether the showtime is valid.

6.3.9 Seat

The `Seat` class represents a seat inside an auditorium.

Attributes

`seatID: int`
Stores the unique ID for the seat.

`auditoriumID: int`
Stores the auditorium that contains the seat.

`row: string`
Stores the row of the seat.

`seatNumber: int`
Stores the number of the seat.

Operations

`isAvailable(showtimeID: int): bool`
Checks whether the seat is available for a selected showtime.

6.3.10 ConcessionItem

The ConcessionItem class represents a food or drink item available at a theater.

Attributes

`itemID: int`
Stores the unique ID for the item.

`theaterID: int`
Stores the theater that sells the item.

`name: string`
Stores the item name.

`price: float`
Stores the price of the item.

`available: bool`
Stores whether the item is currently available.

Operations

`isAvailable(): bool`
Checks whether the item is available for purchase.

`getPrice(): float`
Returns the price of the item.

6.3.11 Order

The `Order` class represents everything the customer is purchasing in one transaction.

Attributes

`orderID: int`
Stores the unique ID for the order.

`customerID: int`
Stores the ID of the customer who created the order.

`tickets: List<Ticket>`
Stores the tickets included in the order.

`concessions: List<ConcessionItem>`
Stores the concession items included in the order.

`totalPrice: float`
Stores the total price of the order.

`status: string`
Stores the current status of the order.

Operations

`addTicket(ticket: Ticket): void`
Adds a ticket to the order.

`addConcession(item: ConcessionItem): void`
Adds a concession item to the order.

`removeTicket(ticketID: int): void`
Removes a ticket from the order.

`removeConcession(itemID: int): void`
Removes a concession item from the order.

`calculateTotal(): float`
Calculates and returns the total price of the order.

`completeOrder(): bool`
Attempts to complete the order and returns whether it was successful.

6.3.12 Ticket

The `Ticket` class represents a purchased seat for a specific showtime.

Attributes

`ticketID: int`
Stores the unique ID for the ticket.

`showtimeID: int`
Stores the showtime connected to the ticket.

`seatID: int`
Stores the seat connected to the ticket.

`price: float`
Stores the price of the ticket.

`status: string`
Stores the current status of the ticket.

Operations

`generateTicket(): string`
Returns the ticket information after a successful purchase.

6.3.13 Payment

The `Payment` class represents the payment made for an order.

Attributes

`paymentID: int`
Stores the unique ID for the payment.

`orderID: int`
Stores the order connected to the payment.

`amount: float`
Stores the payment amount.

`paymentMethod: string`
Stores the payment method used, such as credit card, debit card, or PayPal.

`status: string`
Stores the current payment status.

Operations

`processPayment(): bool`
Attempts to process the payment and returns whether it was successful.

`refund(amount: float): bool`
Attempts to refund the specified amount and returns whether the refund was successful.

6.3.14 Class Relationships

The `Customer` and `Administrator` classes inherit from the `User` class. A `Theater` can contain multiple `Auditorium` objects and can offer multiple `ConcessionItem` objects. A `Movie` can have multiple `Showtime` objects, while each showtime belongs to one movie and one auditorium. An auditorium can contain multiple seats.

A customer can create multiple orders. Each order can contain multiple tickets and concession items. Each order is connected to a payment. A ticket is connected to one showtime and one seat. These relationships allow the system to keep track of what movie the customer is seeing, where the movie is being shown, what seat was purchased, and what other items were included in the order.

6.4 Development Plan and Timeline

The development of the Movie Ticketing System will be divided among the team members based on the main parts of the project. The first part of development will focus on finalizing the design, database structure, and class relationships. After the basic structure is complete, the team will begin implementing the customer and administrator features.

Movie and theater browsing, showtimes, seating, concessions, account management, and checkout will be developed as separate parts of the application. These parts will then be connected and tested together. The final stage of development will focus on fixing problems, improving the interface, and completing the project documentation.

6.4.1 Task Partitioning

Task	Team Member	Estimated Time
Review requirements and design	All members	Week 1
Software architecture diagram	Cameron Penaranda	Week 1
UML class diagram	Justin Paul Liporada	Week 1
Database design	Ayaan Bhatkar	Week 1
Login and account system	Cameron Penaranda	Week 2
Movie and theater browsing	Justin Paul Liporada	Week 2
Showtime and seat selection	Ayaan Bhatkar	Week 2 to 3
Food and drink ordering	Justin Paul Liporada	Week 3
Checkout and payment	Cameron Penaranda	Week 3 to 4
Database integration	Ayaan Bhatkar	Week 3 to 4
System testing	All members	Week 5
Bug fixes and final changes	All members	Week 6
Final documentation	All members	Week 6

6.4.2 Team Member Responsibilities

Cameron Penaranda will mainly work on the software architecture, account system, and checkout and payment functionality. Justin Paul Liporada will mainly work on the UML design, movie and theater browsing, and food and drink functionality. Ayaan Bhatkar will mainly work on the database, showtime functionality, seat selection, and database integration.

All team members will help with testing, reviewing changes, fixing bugs, and updating the project documentation. Each team member will also make commits to the project GitHub repository throughout development. All team members will also incorporate the newly added Support Agents interface to wherever it applies.

6.5 Data Management Strategy

The Movie Ticketing System requires persistent storage for customer accounts, movie information, theater information, seat availability, purchases, payments, and customer support information. The data management strategy is designed to maintain accurate relationships between these entities while protecting sensitive information.

The system uses a relational database approach because many of the system's functions depend on connected data. For example, customers create orders, orders contain tickets, and tickets reference specific seats and showtimes. Maintaining these relationships is important for preventing incorrect purchases and keeping information consistent.

6.5.1 Database Selection

The Movie Ticketing System will use PostgreSQL, a relational SQL database. The system contains information that is closely connected. Customers create orders, orders contain tickets, and each ticket identifies a seat for a specific showtime. A relational database allows these connections to be represented through tables and foreign keys.

The system will use one operational database for all participating theaters. This keeps customer accounts, theater searches, and purchases in one place. It also allows related changes, such as confirming an order and its tickets, to be completed within the same database transaction. Using one database reduces the setup and maintenance needed for the project.

Separate database copies will be used for development and testing with sample data. These copies and backup files are separate from the database used by the deployed application.

6.5.2 Logical Organization of Data

The database will contain four logical groups of tables. These groups organize related information within the same database while maintaining relationships between connected data.

Group	Tables	Information stored
Accounts	Users	User IDs, usernames, email addresses, password hashes, and account roles
Catalog and seating	Movies, Theaters, Auditoriums, Seats, Showtimes, ConcessionItems	Movie information, theater locations, auditorium layouts, scheduled showtimes, and theater-specific food and drink offerings
Purchases and reservations	Orders, Tickets, OrderConcessions, Payments	Customer orders, temporary seat holds, confirmed tickets, concession quantities, purchase prices, and payment attempts

Group	Tables	Information stored
Customer support	SupportTickets, ChatMessages	Customer questions, assigned support agents, ticket statuses, and conversation history

Customer, administrator, and support agent accounts will share the Users table. The account role will determine which application functions each user can access.

Each auditorium will belong to a theater, and each seat will belong to an auditorium. Each showtime will reference a movie and an auditorium. An order will belong to a customer, and its tickets will use the selected showtime.

OrderConcessions will connect orders to concession items and store the quantity and purchase-time unit price. Payments will store payment attempts, including amount, payment method, provider reference, and status. SupportTickets and ChatMessages will be stored separately from admission tickets.

6.5.3 Data Integrity and Storage Rules

Each table will have a primary key that uniquely identifies its records. Foreign keys will maintain valid relationships between tables. For example, a ticket must reference an existing order, showtime, and seat.

Monetary values will use fixed decimal storage, such as NUMERIC(10,2), instead of floating-point values. Ticket prices and concession prices will be saved when purchased so that later price changes do not affect previous receipts. Ordered quantities must be positive integers, and prices cannot be negative.

Seat availability will be tracked using the combination of showtimeID and seatID. The database will prevent more than one active held or confirmed ticket for that combination. A physical seat can still be purchased for different showtimes.

The application will also check that a selected seat belongs to the auditorium used by the showtime and that concession items belong to the selected theater. Movies, showtimes, and concession items referenced by purchase history will be marked inactive instead of being deleted.

Dates and times will be stored consistently in UTC and displayed using the theater’s local time zone. Indexes will support frequently used searches, including showtimes by movie, orders by customer, and tickets by showtime.

6.5.4 Seat Reservations and Payment Processing

The first seat selected for an order will start a 20-minute hold. Adding more seats will not restart the timer. The deadline will be stored with the order so that it remains available if the customer refreshes the page.

The application will check availability and save requested seat holds within one transaction. If another customer has already reserved a requested seat, the transaction will fail and the customer will be asked to select another seat. Expired holds will be released by a scheduled cleanup process.

Before payment, the application will recheck the seat hold, concession availability, and order total. A pending payment attempt will be created before contacting the payment provider.

After successful payment authorization, the application will confirm the order and tickets within a database transaction. Declined payments will be marked as failed, while uncertain payment results will remain pending until verified.

6.5.5 Security and Access Control

Passwords will be stored as salted password hashes rather than readable text. Full payment-card numbers and card security codes will not be stored. The payment provider will handle sensitive payment information while the application stores only transaction references.

Customers will only be allowed to access their own orders and support conversations. Administrators will manage movie, showtime, and concession information. Support agents will access customer information necessary to resolve assigned support requests.

The application will use HTTPS and parameterized database queries. Database credentials will remain in server configuration and will not be committed to GitHub. Customer location coordinates will only be used for theater searches and will not be stored as location history.

6.5.6 Backup, Recovery, and Retention

The proposed backup policy is a daily encrypted database backup with continuous archiving of PostgreSQL transaction logs. Backups and required logs will be stored separately from the database server and restricted to authorized maintainers.

Recovery will restore a backup and apply available transaction logs. Restore procedures will be tested regularly to ensure that the system can recover within the availability requirement.

Completed orders and payment records will be retained to support purchase history and refund questions. Closed support tickets and chat messages will be retained for customer service purposes. Unnecessary information will be removed or anonymized when it is no longer required.

6.5.7 Alternatives and Tradeoffs

Alternative	Benefits	Tradeoffs
SQLite	Simple setup and useful for a small local prototype.	It permits one writer at a time per database file. A database server is a better fit for the intended application with concurrent purchases.
MongoDB	Flexible documents could work well for catalog information or chat. It also supports transactions across documents.	The system still needs to coordinate customers, orders, seats, showtimes, and payments. Related SQL tables make these connections easier to organize for this project.
A separate database for each theater or component	Allows more independent maintenance and scaling.	Shared accounts, searches across theaters, backups, and related purchase updates would require additional coordination.
One large table containing all information	Simple to create initially and requires fewer joins.	Repeats customer, movie, and theater information. Updating repeated values can leave inconsistent records.

The selected PostgreSQL design provides the best balance between consistency, maintainability, and system requirements. The Movie Ticketing System depends heavily on relationships between customers, purchases, tickets, seats, and showtimes, making a relational database the most appropriate choice.

The main disadvantage of the selected approach is that all components share database resources, meaning a database failure could affect the entire application. If the system grows significantly in the future, separate databases or additional infrastructure could be considered.

7. Test Plan

This section defines the verification and validation testing plan for the Movie Ticketing System. The purpose of testing is to verify that the implemented features satisfy the functional and non-functional requirements defined in this Software Requirements Specification (SRS) and validate that the system supports the expected workflows for customers, administrators, and support agents. Testing will be performed throughout development to identify software defects, verify correct interaction between software components, and ensure the completed system performs reliably under both normal and abnormal operating conditions. The testing process includes unit, functional, integration, and system-level testing to provide comprehensive coverage of the software system.

7.1 Testing Objectives

The primary objective of testing is to verify that every major feature of the Movie Ticketing System operates according to the requirements defined in this Software Requirements Specification. Testing will ensure that customer functions such as browsing movies, selecting theaters and showtimes, reserving seats, purchasing tickets, ordering concessions, and receiving purchase confirmations perform correctly. Testing will also verify administrator functions including managing movies, showtimes, theaters, and customer support requests.

The testing process is designed to identify software defects before deployment, verify that data is processed correctly throughout the system, validate communication between software components, and confirm that invalid inputs or unexpected situations are handled appropriately. Successful testing demonstrates that the Movie Ticketing System satisfies the required functionality, maintains data integrity, and provides a reliable user experience.

7.2 Verification and Validation Strategy

Verification testing confirms that individual software components have been implemented according to the software design specification and UML class design. Individual classes, methods, and functions will be tested independently before being integrated into the complete system. Examples include verifying that the Order.calculateTotal() method correctly calculates purchase totals, the Seat.isAvailable() method accurately determines seat availability, and the Payment.processPayment() method correctly accepts or rejects payment transactions.

Validation testing confirms that the completed Movie Ticketing System satisfies user requirements by testing complete customer and administrator workflows. Validation scenarios simulate realistic system usage, including browsing available movies, selecting theaters and showtimes, reserving seats, purchasing tickets, ordering concession items, and receiving purchase confirmations. Administrator validation includes managing movie listings, updating showtimes, and responding to customer support requests.

Feature Tested	Test Input (Test Vector)	Expected Result	Failure Condition
Browse Movies	Customer selects "Browse Movies"	Current movies are displayed with title, rating, runtime, and description	Movie list fails to load or displays incorrect information
Seat Selection	Customer selects Seat A10	Seat becomes temporarily reserved and unavailable to other users	Seat can still be selected by another customer
Ticket Purchase	Customer purchases two tickets using a valid payment method	Payment succeeds, order is stored, confirmation is generated	Payment succeeds but ticket is not generated
Invalid Payment	Customer enters an expired credit card	Payment is rejected and order is cancelled	Invalid payment is accepted

7.3 Unit Testing

Unit testing verifies that individual classes and methods perform correctly in isolation before integration with the remainder of the system. Each method will be tested using both valid and invalid inputs to verify correct functionality and proper error handling.

Order.calculateTotal()

- **Test Input:** Adult Ticket = $12.00, Child Ticket = $8.00, Popcorn = $6.00
- **Expected Output:** Total purchase price = $26.00
- **Failure Conditions:** Incorrect total calculation, duplicate item calculation, missing concession item, incorrect tax or subtotal calculation.

Seat.isAvailable(showtimeID)

- Test Input: Seat A10 marked as available.
- Expected Output: Method returns True.
- Failure Conditions: Returns False for an available seat or True for a seat that has already been reserved.

Payment.processPayment()

- Test Input: Valid payment information.
- Expected Output: Payment approved and transaction recorded.
- Failure Conditions: Valid payment rejected or transaction not stored.

7.4 Functional Testing

Functional testing verifies that the Movie Ticketing System satisfies the functional requirements defined in the Software Requirements Specification by evaluating complete user-facing features. Each functional test focuses on a specific requirement and verifies both expected behavior and error handling.

Feature	Test Input	Expected Result	Failure Condition
Browse Movies	Customer opens Browse Movies page	Current movies displayed with complete information	Movies fail to display or contain incorrect information
View Showtimes	Customer selects a movie	Available showtimes displayed	Missing or incorrect showtimes
Select Seats	Customer selects available seats	Selected seats become reserved	Already reserved seats remain selectable
Purchase Tickets	Customer submits valid payment	Purchase completes and confirmation generated	Payment accepted without generating ticket
Browse Concessions	Customer opens concession menu	Available concession items displayed	Menu missing or incomplete

Administrator Adds Movie	Administrator submits new movie information	Movie successfully added to system	Movie not stored or duplicate information created

7.5 System Testing / Integration testing

Integration testing verifies that multiple software components communicate correctly and exchange information without data loss or processing errors. Testing focuses on the interaction between the Order, Payment, Ticket, Seat, Movie, Showtime, and database components.

Representative integration tests include:

- Verify that successful payment automatically generates a ticket.
- Verify that completed purchases update seat availability in the database.
- Verify that cancelled purchases release reserved seats.
- Verify that concession purchases are included in the final order total.

System testing evaluates complete end-to-end workflows that simulate real-world usage of the Movie Ticketing System.

7.5.1 Scenario 1 – Complete Ticket Purchase

Test Vector

1. Customer browses available movies.
2. Customer selects a theater and showtime.
3. Customer reserves Seats A5 and A6.
4. Customer adds popcorn and a drink.
5. Customer submits valid payment information.

Expected Results

- Payment is successfully processed.
- Seats become unavailable.
- Order information is stored.
- Electronic ticket is generated.
- Purchase confirmation is displayed.

Failure Conditions

- Payment succeeds without generating a ticket.
- Seat reservation is not updated.

- Confirmation is not displayed.
- Duplicate orders are created.

7.5.2 Scenario 2 – Double Seat Reservation

Test Vector

Two customers attempt to purchase Seat A8 for the same showtime simultaneously.

Expected Results

- First purchase succeeds.
- Second purchase is rejected because the seat is no longer available.

Failure Conditions

- Both customers successfully purchase the same seat.
- Seat availability is not updated correctly.

7.6 Test Coverage

Testing covers all major software components and functional requirements identified within the Movie Ticketing System. Test cases have been developed to verify both normal operation and failure scenarios for customer, administrator, and support agent functionality.

Requirement	Test Case(s)
Browse Movies	TC-003
View Showtimes	TC-004
Seat Availability	TC-002, TC-007
Ticket Purchase	TC-006
Order Total Calculation	TC-001
Payment Processing	TC-006, TC-008
Administrator Movie Management	TC-005
Browse Concessions	TC-009
Customer Support Tickets	TC-010

7.7 Test Environment

Testing will be performed using the completed Movie Ticketing System application within the project's development environment. The test environment includes the application user interface, application logic, database, and simulated payment processing services where required. Testing will be performed using representative customer, administrator, and support agent accounts with sample movie listings, theaters, showtimes, seat inventories, concession items, and payment information. This environment provides sufficient coverage for verifying both normal operation and common failure scenarios without affecting production data.

7.8 Test Case Document

The Movie Ticketing System includes ten detailed test case samples developed using the provided Excel Test Case template. These test cases include unit, functional, and system-level testing and contain the required preconditions, test inputs, execution steps, expected outputs, actual results, and execution status.

The detailed test cases verify the core functionality of the Movie Ticketing System, including order total calculation, seat availability, movie browsing, showtime selection, administrator movie management, complete ticket purchasing workflows, payment validation, concession ordering, and customer support ticket submission.

Table 7.1 summarizes the implemented test cases, while the complete test case documentation is provided in the accompanying Excel workbook.

Test Case Template							
TC-001	**Unit**	**Calculate Order Total**	**Order.calculateTotal()**	**Order contains**	**Ticket=$12, Popcorn=$5**	**Create order and**	**Returns total price of $17**
TC-002	Unit	Check Seat Availability	Seat.isAvailable(showtimeID)	Seat exists for selected showtime	Seat A1 reserved/unreserved states	Call availability check	Returns true for available seats and false for reserved seats
TC-003	Functional	Browse Available Movies	Browse Movies requirement	Movie database	Customer requests movie list	Open movie brow	System displays available movies with information
TC-004	Functional	Select Available Showtime	View Showtimes requirement	Movie has sche	Select a movie	Retrieve showtim	System displays valid dates and start times
TC-005	Functional	Administrator Adds Movie	Manage Movies requirement	Administrator a	New movie information	Submit add movi	Movie is validated and added to system
TC-006	System	Complete Ticket Purchase Flow	Purchase Tickets and Confirma	Customer has s	Valid credit card payment	Browse movie, se	Order completes and confirmation is generated
TC-007	System	Prevent Double Seat Booking	Seat reservation reliability req	Two customers	Same seat for same showtime	Customer A reser	Second customer cannot purchase unavailable seat
TC-008	System	Handle Failed Payment	Payment failure handling	Customer has s	Invalid payment information	Attempt checkout	Payment fails and ticket is not created
TC-009	Functional	Browse Concession Items	Browse Food and Drink require	Theater has ava	Selected theater	Open concession	Items and prices are displayed and can be added
TC-010	Functional	Customer Support Ticket Submissio	Support Tickets and Customer	Customer is log	Support issue description	Submit support ti	Ticket is stored and visible to support agents

7.9 GitHub Repository

Repository Link: https://github.com/cameronpenaranda/cs250-movie-ticketing-system

Each group member is responsible for contributing at least one commit to the repository prior to each group submission. The repository serves as the central location for project documentation, version control, and assignment deliverables.

A. Appendices

No supplementary material is included in this revision. Future appendices will state whether their content is part of the system requirements.

A.1 Appendix 1

A.2 Appendix 2